

UNIT – 4

LEETCODE EXERCISE

1. Delete Duplicate Emails

Table: Person

Column Name	Type
Id	int
email	varchar

id is the primary key (column with unique values) for this table.

Each row of this table contains an email. The emails will not contain uppercase letters.

Problem Summary

Write a solution to **delete** all duplicate emails, keeping only one unique email with the smallest id.

For SQL users, please note that you are supposed to write a DELETE statement and not a SELECT one.

For Pandas users, please note that you are supposed to modify Person in place.

After running your script, the answer shown is the Person table. The driver will first compile and run your piece of code and then show the Person table. The final order of the Person table **does not matter**.

SQL Schema

```
CREATE TABLE If Not Exists Person (Id int, Email varchar(255))
```

```
TRUNCATE TABLE Person
```

```
INSERT INTO Person (id, email) values ('1', 'john@example.com')
```

```
INSERT INTO Person (id, email) values ('2', 'bob@example.com')
```

```
INSERT INTO Person (id, email) values ('3', 'john@example.com')
```

Person

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

MySQL Solution

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 delete p1
3 from Person p1, Person p2
4 where p1.Email = p2.Email and p1.Id > p2.Id
```

Output

```
| id | email          |
| ---|-----|
| 1  | john@example.com |
| 2  | bob@example.com  |
```

2. Customer who never order

Table: Customer

Column Name	Type
id	int
name	varchar

id is the primary key (column with unique values) for this table.

Each row of this table indicates the ID and name of a customer.

Table : Order

Column Name	Type
id	int
customerId	varchar

id is the primary key (column with unique values) for this table.

customerId is a foreign key (reference columns) of the ID from the Customers table.

Each row of this table indicates the ID of an order and the ID of the customer who ordered it.

Problem Summary

Write a solution to find all customers who never order anything.

Return the result table in **any order**.

SQL Schema

```
CREATE TABLE If Not Exists Customers (Id int, Name varchar(255));
```

```
CREATE TABLE If Not Exists Orders (Id int, CustomerId int);
```

```
TRUNCATE TABLE Customers;
```

```
insert into Customers (Id, Name) values
```

```
('1', 'Joe'),
```

```
('2', 'Henry'),
```

```
('3', 'Sam'),
```

```
('4', 'Max');
```

```
Truncate table Orders;
```

```
insert into Orders (Id, CustomerId) values
```

```
('1', '3'),
```

```
('2', '1');
```

Customer

Id	Name
1	Joe
2	Henry
3	Sam
4	Max

Order

Id	Customer Id
1	Joe
2	Henry
3	Sam

MySQL Solution

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select c.Name as Customers
3 from Customers c
4 where c.Id not in (select CustomerId from Orders);
```

Output

```
| Customers |
|-----|
| Henry    |
| Max      |
```

3. Biggest Single Numbers

Table: MyNumbers

Column Name	Type
num	int

This table may contain duplicates (In other words, there is no primary key for this table in SQL).

Each row of this table contains an integer.

Problem Summary

A **single number** is a number that appeared only once in the MyNumbers table.

Find the largest **single number**. If there is no **single number**, report null.

SQL Schema

```
CREATE TABLE If Not Exists my_numbers (num int)
```

```
Truncate table my_numbers
```

```
insert into my_numbers (num) values ('8')
```

```
insert into my_numbers (num) values ('8')
```

```
insert into my_numbers (num) values ('3')
```

```
insert into my_numbers (num) values ('3')
```

```
insert into my_numbers (num) values ('1')
```

```
insert into my_numbers (num) values ('4')
```

```
insert into my_numbers (num) values ('5')
```

```
insert into my_numbers (num) values ('6')
```

Visits

num
8
8
3
3
1
5
6

MySQL Solution

`</>` Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select max(num) as num
3 from (
4     select num
5     from MyNumbers
6     group by num
7     having count(num) = 1
8 ) n;
```

Output

```
| num |
| --- |
| 6   |
```

4. Game Play Analysis I

Table: Activity

Column Name	Type
primary_Id	int
device_Id	int
Event_date	date
games_played	int

(player_id, event_date) is the primary key (combination of columns with unique values) of this table.

This table shows the activity of players of some games.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.

Problem Summary

Write a solution to find the **first login date** for each player.

Return the result table in **any order**..

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int)

Truncate table Activity

```
insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5')
```

```
insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6')
```

```
insert into Activity (player_id, device_id, event_date, games_played) values ('2', '3', '2017-06-25', '1')
```

```
insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0')
```

```
insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5')
```

Acitivity

Player_id	device_id	Event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

Result table

player id	first login
1	2016-03-01
2	2017-06-25
3	2016-03-02

MySQL Solution

```
/* MySQL Query: */
select a.player_id, min(a.event_date) as first_login
from Activity a
group by a.player_id
```

Output

```
| player_id | first_login |
| ----- | -
```

5. Game Play Analysis II

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key of this table.

This table shows the activity of players of some game.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on some day using some device.

Problem Summary

Write a SQL query that reports the device that is first logged in for each player.

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int)

Truncate table Activity

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5')

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6')

insert into Activity (player_id, device_id, event_date, games_played) values ('2', '3', '2017-06-25', '1')

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0')

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5')

Activity table:

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

MySQL Solution

```
/* MySQL Query */
select player_id, device_id
from Activity
where (player_id, event_date) in
>   (select player_id, min(event_date)
    from Activity
    group by player_id);
```

Result table:

player_id	device_id
1	2
2	3
3	1

5. Game Play Analysis III

Table: Activity

Column Name	Type
player_id	int
device_id	int
event_date	date
games_played	int

(player_id, event_date) is the primary key (column with unique values) of this table.

This table shows the activity of players of some games.

Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on someday using some device.

Problem Summary

Write a solution to report for each player and date, how many games played so far by the player. That is, the total number of games played by the player until that date. Check the example for clarity.

SQL Schema

Create table If Not Exists Activity (player_id int, device_id int, event_date date, games_played int)

Truncate table Activity

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-03-01', '5')

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '2', '2016-05-02', '6')

insert into Activity (player_id, device_id, event_date, games_played) values ('1', '3', '2017-06-25', '1')

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '1', '2016-03-02', '0')

insert into Activity (player_id, device_id, event_date, games_played) values ('3', '4', '2018-07-03', '5')

Activity table:

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

MySQL Solution

```
/* MySQL Query*/
SELECT
    a.player_id,
    a.event_date,
    SUM(b.games_played) games_played_so_far
FROM
    Activity a LEFT JOIN Activity b ON
    b.player_id = a.player_id AND
    b.event_date <= a.event_date
GROUP BY
    a.player_id,
    a.event_date;
```

Output:

player_id	event_date	games_played_so_far
1	2016-03-01	5
1	2016-05-02	11
1	2017-06-25	12
3	2016-03-02	0
3	2018-07-03	5

6. Exchange Seat

Table: Activity

Column Name	Type
id	int
student	varchar

id is the primary key (unique value) column for this table.

Each row of this table indicates the name and the ID of a student.

The ID sequence always starts from 1 and increments continuously.

Problem Summary

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.

Return the result table ordered by id in ascending order.

SQL Schema

Create table If Not Exists Seat (id int, student varchar(255))

Truncate table Seat

insert into Seat (id, student) values ('1', 'Abbot')

insert into Seat (id, student) values ('2', 'Doris')

insert into Seat (id, student) values ('3', 'Emerson')

insert into Seat (id, student) values ('4', 'Green')

insert into Seat (id, student) values ('5', 'Jeames')

Seat table

id	student
1	Abbot
2	Doris
3	Emerson
4	Green
5	Jeames

MySQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     a.id id,
4     COALESCE(b.student, a.student) student
5 FROM
6     seat a LEFT JOIN seat b
7     ON b.id = (CASE
8         WHEN a.id%2=1 THEN a.id+1
9         ELSE a.id-1
10        END)
11 ORDER BY a.id
```

Output

id	student
1	Doris
2	Abbot
3	Green
4	Emerson
5	Jeames

27. TRIANGLE JUDGEMENT

Table: Triangle

Column Name	Type
x	int
y	int
z	int

In SQL, (x, y, z) is the primary key column for this table.
Each row of this table contains the lengths of three line segments.

Problem Summary

Report for every three line segments whether they can form a triangle.

Return the result table in **any order**.

SQL Schema

Create table If Not Exists triangle (x int, y int, z int)

Truncate table triangle

insert into triangle (x, y, z) values ('13', '15', '30')

insert into triangle (x, y, z) values ('10', '20', '15')

Triangle table:

x	y	z
13	15	30
10	20	15

MySQL Solution

Write your MySQL query statement below

```
select *,  
       if(x+y>z and x+z>y and y+z>x, 'Yes', 'No') as triangle  
from triangle
```

Output

x	y	z	triangle
13	15	30	No
10	20	15	Yes

8. Consecutive Numbers

Table: Logs

Column Name	Type
id	int
num	varchar

In SQL, id is the primary key for this table.
id is an autoincrement column starting from 1.

Problem Summary

Find all numbers that appear at least three times consecutively.

Return the result table in **any order**.

SQL Schema

Create table If Not Exists Logs (id int, num int);

Truncate table Logs;

insert into Logs (id, num) values ('1', '1');

insert into Logs (id, num) values ('2', '1');

insert into Logs (id, num) values ('3', '1');

insert into Logs (id, num) values ('4', '2');

insert into Logs (id, num) values ('5', '1');

insert into Logs (id, num) values ('6', '2');

insert into Logs (id, num) values ('7', '2');

Logs table:

id	num
1	1
2	1
3	1
4	2
5	1
6	2
7	2

MySQL Solution

Write your MySQL query statement below

```
SELECT DISTINCT L1.NUM ConsecutiveNums
FROM Logs l1, Logs l2, LOGS L3
WHERE l1.Id = l2.Id - 1
      AND l2.ID=L3.ID - 1
      AND l1.Num = l2.Num
      AND L2.NUM = L3.NUM;
```

Output

```
| ConsecutiveNums |
| ----- |
| 1               |
```

9. TREE NODE

Table: Tree

Column Name	Type
id	int
p_id	int

id is the column with unique values for this table.

Each row of this table contains information about the id of a node and the id of its parent node in a tree.

The given structure is always a valid tree.

Each node in the tree can be one of three types:

- **"Leaf"**: if the node is a leaf node.
- **"Root"**: if the node is the root of the tree.
- **"Inner"**: If the node is neither a leaf node nor a root node.

Problem Summary

Write a solution to report the type of each node in the tree.

Return the result table in **any order**.

SQL Schema

```
Create table If Not Exists Tree (id int, p_id int);
```

```
Truncate table Tree;
```

```
insert into Tree (id, p_id) values ('1', NULL);
```

```
insert into Tree (id, p_id) values ('2', '1');
```

```
insert into Tree (id, p_id) values ('3', '1');
```

```
insert into Tree (id, p_id) values ('4', '2');
```

```
insert into Tree (id, p_id) values ('5', '2');
```

Tree table

id	p_id
1	Alice
2	Bob
13	John
6	Alex

MySQL Solution

Write your MySQL query statement below

```
SELECT
  id,
  (CASE
    WHEN p_id IS NULL THEN "Root"
    WHEN id IN (SELECT DISTINCT p_id FROM tree) THEN "Inner"
    ELSE "Leaf"
  END) Type
FROM tree;
```

Output

```
| id | Type |
| --- | --- |
| 1 | Root |
| 2 | Inner |
| 3 | Leaf |
| 4 | Leaf |
| 5 | Leaf |
```